

Teach Module 49: Capstone Build - Backend and Messaging

This note is based on Module 49's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

Module 49 Overview

Module 49 is about building the backend and event-driven messaging parts of a capstone application. The backend receives API requests, applies business rules, persists data, and publishes events. Messaging lets other parts of the system react asynchronously when something important happens.

The main topics are:

- Java and Spring Boot backend services
- Kafka producer and consumer integration
- Service layer and repository implementation
- Unit and integration testing
- Internal code review and pair programming
- Capstone backend and messaging lab

Big Picture

A common backend flow looks like this:

```
Client
-> REST Controller
-> Service Layer
-> Repository
-> Database
-> Kafka Producer
-> Kafka Topic
-> Kafka Consumer
-> Follow-up action
```

For example, in an order management system:

```
POST /orders
-> validate request
-> create order
-> save order
-> publish OrderCreatedEvent
-> notification service consumes event
-> notification record is created
```

The important idea is that the backend does not only respond to API calls. It can also announce important business events so other parts of the system can react.

Spring Boot Backend Services

A Spring Boot backend usually separates responsibilities into layers:

```
Controller -> Service -> Repository -> Database
```

The controller handles HTTP requests and responses.

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public OrderResponse createOrder(@RequestBody CreateOrderRequest request) {
        return orderService.createOrder(request);
    }
}
```

The controller should stay thin. It should not contain the main business logic. Its job is to receive input, call the service, and return the result.

Service Layer

The service layer contains business rules and workflow decisions.

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final OrderEventProducer eventProducer;

    public OrderService(OrderRepository orderRepository,
                        OrderEventProducer eventProducer) {
        this.orderRepository = orderRepository;
        this.eventProducer = eventProducer;
    }

    public OrderResponse createOrder(CreateOrderRequest request) {
        if (request.customerId() == null) {
            throw new IllegalArgumentException("Customer ID is required");
        }

        if (request.quantity() == null || request.quantity() <= 0) {
            throw new IllegalArgumentException("Quantity must be greater than zero");
        }
    }
}
```

```

    Order order = new Order();
    order.setCustomerId(request.customerId());
    order.setProductName(request.productName());
    order.setQuantity(request.quantity());
    order.setStatus("CREATED");
    order.setCreatedAt(LocalDateTime.now());

    Order savedOrder = orderRepository.save(order);

    eventProducer.publishOrderCreated(savedOrder);

    return new OrderResponse(
        savedOrder.getId(),
        savedOrder.getCustomerId(),
        savedOrder.getProductName(),
        savedOrder.getQuantity(),
        savedOrder.getStatus(),
        savedOrder.getCreatedAt()
    );
}

```

This service does four useful things:

1. Validates the request.
2. Creates the domain object.
3. Saves it through the repository.
4. Publishes an event after the save succeeds.

Repository Layer

The repository handles database access. With Spring Data JPA, a repository can be created by extending `JpaRepository`.

```

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByCustomerId(Long customerId);
}

```

Spring Data JPA automatically provides common methods:

```

save()
findById()
findAll()
deleteById()

```

The repository should not contain business decisions. Its job is persistence.

Kafka Producer

A Kafka producer publishes messages to a Kafka topic. In a backend application, a producer is often used after a meaningful state change.

Example:

```
Order is created -> publish OrderCreatedEvent
Payment is approved -> publish PaymentApprovedEvent
Ticket is assigned -> publish TicketAssignedEvent
```

Producer example:

```
@Service
public class OrderEventProducer {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishOrderCreated(Order order) {
        OrderCreatedEvent event = new OrderCreatedEvent(
            order.getId(),
            order.getCustomerId(),
            order.getProductName(),
            order.getQuantity(),
            order.getStatus(),
            order.getCreatedAt()
        );

        kafkaTemplate.send("order-created", String.valueOf(order.getId()), event);
    }
}
```

The topic name should be clear and event-based:

```
order-created
order-status-changed
payment-approved
inventory-reserved
```

The message key should usually be something stable, such as an order ID, customer ID, or account ID. This helps Kafka keep related messages ordered within a partition.

Kafka Consumer

A Kafka consumer listens to a topic and reacts when a message arrives.

```
@Service
public class OrderCreatedConsumer {

    private final NotificationRepository notificationRepository;
```

```

public OrderCreatedConsumer(NotificationRepository notificationRepository) {
    this.notificationRepository = notificationRepository;
}

@KafkaListener(topics = "order-created", groupId = "notification-service")
public void handleOrderCreated(OrderCreatedEvent event) {
    Notification notification = new Notification();
    notification.setOrderId(event.orderId());
    notification.setMessage("Order created for product: " + event.productName());
    notification.setCreatedAt(LocalDateTime.now());

    notificationRepository.save(notification);
}
}

```

A producer says:

Something happened.

A consumer says:

I care that this happened, so I will react.

That is the heart of event-driven backend design.

Why Messaging Helps

Without messaging, one service may directly call many other services:

```

Order Service -> Notification Service
Order Service -> Billing Service
Order Service -> Shipping Service

```

This can create tight coupling. If one downstream service is slow or unavailable, the main request flow may suffer.

With messaging:

```

Order Service -> Kafka topic -> Other services consume when ready

```

Messaging helps with:

- Loose coupling
- Scalability
- Fault tolerance
- Asynchronous workflows
- Event history
- Cleaner boundaries between services

DTOs And Events

Use DTOs for API input and output. Use event objects for Kafka messages.

Request DTO:

```
public record CreateOrderRequest(  
    Long customerId,  
    String productName,  
    Integer quantity  
) {  
}
```

Response DTO:

```
public record OrderResponse(  
    Long id,  
    Long customerId,  
    String productName,  
    Integer quantity,  
    String status,  
    LocalDateTime createdAt  
) {  
}
```

Event DTO:

```
public record OrderCreatedEvent(  
    Long orderId,  
    Long customerId,  
    String productName,  
    Integer quantity,  
    String status,  
    LocalDateTime createdAt  
) {  
}
```

Avoid exposing JPA entities directly from controllers. Entities are database-focused. DTOs are API-focused. Events are messaging-focused.

Testing

Testing should cover both business behavior and system integration.

Unit Testing

A unit test checks one class in isolation. For example, test `OrderService` without a real database or Kafka broker.

```
@ExtendWith(MockitoExtension.class)  
class OrderServiceTest {  
  
    @Mock
```

```

private OrderRepository orderRepository;

@Mock
private OrderEventProducer eventProducer;

@InjectMocks
private OrderService orderService;

@Test
void createOrder_savesOrderAndPublishesEvent() {
    CreateOrderRequest request = new CreateOrderRequest(100L, "Laptop", 1);

    Order savedOrder = new Order();
    savedOrder.setId(1L);
    savedOrder.setCustomerId(100L);
    savedOrder.setProductName("Laptop");
    savedOrder.setQuantity(1);
    savedOrder.setStatus("CREATED");
    savedOrder.setCreatedAt(LocalDate.now());

    when(orderRepository.save(any(Order.class))).thenReturn(savedOrder);

    OrderResponse response = orderService.createOrder(request);

    assertEquals(1L, response.id());
    assertEquals("CREATED", response.status());

    verify(orderRepository).save(any(Order.class));
    verify(eventProducer).publishOrderCreated(savedOrder);
}
}

```

Good unit tests for this module should verify:

- Valid data creates the entity.
- Invalid data is rejected.
- The repository is called.
- The producer is called.
- The response contains expected values.

Integration Testing

An integration test checks whether multiple parts work together.

Examples:

- Controller plus service
- Service plus repository
- API plus database
- Producer plus Kafka
- Consumer plus Kafka

Common tools:

- `@SpringBootTest`
- `@WebMvcTest`
- `MockMvc`
- `Testcontainers`
- Embedded database such as H2

Practice Exercises

Exercise 1: Build A REST Endpoint

Create a Spring Boot API for one capstone entity, such as:

- Order
- Ticket
- Payment
- Booking
- InventoryItem

Implement:

```
POST /orders
GET /orders/{id}
GET /orders
PUT /orders/{id}/status
DELETE /orders/{id}
```

Practice goal: understand controllers, request/response DTOs, and basic service flow.

Exercise 2: Add Service Layer Logic

Move all business rules into a service class.

Example rules:

```
An order cannot be created without a customer ID.
An order starts with status CREATED.
An order can move from CREATED to APPROVED.
An order cannot move from CANCELLED to APPROVED.
```

Practice goal: keep controllers thin and put real application behavior in services.

Exercise 3: Implement Repository Persistence

Use Spring Data JPA with an entity and repository.

Practice goal: store and retrieve backend data using repository patterns.

Exercise 4: Publish A Kafka Event

When a new order is created, publish an event:

```
Topic: order-created
Event: OrderCreatedEvent
```



```
Fields: orderId, customerId, status, createdAt
```

Practice goal: learn how backend actions become messages other systems can consume.

Exercise 5: Consume A Kafka Event

Create a Kafka consumer that listens to `order-created`.

When it receives an event, log:

```
Received order-created event for order 123
```

Then extend it to create a notification record.

Practice goal: understand asynchronous processing.

Exercise 6: Add Error Handling

Add proper API error responses for:

- Invalid request body
- Entity not found
- Invalid status transition
- Kafka publishing failure

Practice goal: make the backend production-minded rather than happy-path only.

Exercise 7: Unit Test The Service Layer

Write tests for the service class using JUnit and Mockito.

Test cases:

- `createOrder` saves the order.
- `createOrder` publishes an event.
- `createOrder` rejects missing customer ID.
- `updateStatus` allows valid status changes.
- `updateStatus` rejects invalid status changes.

Practice goal: test business behavior without starting the full application.

Exercise 8: Integration Test The API

Use `@SpringBootTest` or `@WebMvcTest` to test the API.

Example test cases:

- `POST /orders` returns success.
- `GET /orders/{id}` returns the saved order.
- `GET /orders/{id}` returns 404 for a missing order.

Practice goal: verify that controller, validation, and response mapping work correctly.

Exercise 9: Integration Test Kafka Flow

Use Testcontainers if available.

Test this flow:

```
Create order
Producer sends Kafka event
Consumer receives event
Consumer performs expected action
```

Practice goal: validate the real messaging pipeline.

Exercise 10: Internal Code Review

Review your own code or a partner's code using this checklist:

- Are controllers thin?
- Is business logic in services?
- Are DTOs separate from entities?
- Are repository methods simple?
- Are Kafka topic names clear?
- Are events versionable and readable?
- Are errors handled cleanly?
- Are unit tests meaningful?
- Are integration tests covering the main flow?

Module 49 Lab: Capstone Backend And Messaging

Lab Goal

Build a Spring Boot backend feature that saves an entity and publishes a Kafka event. Then consume that event and perform a follow-up action.

Use this scenario:

```
Order Management System
```

When an order is created:

1. API receives the order request.
2. Backend saves the order.
3. Backend publishes an OrderCreatedEvent to Kafka.
4. Consumer receives the event.
5. Consumer creates a notification record.

What You Will Build

Endpoints:

```
POST /orders
GET /orders/{id}
GET /orders
GET /notifications
```

Kafka topic:

```
order-created
```

Core classes:

```
Order
Notification
CreateOrderRequest
OrderResponse
OrderCreatedEvent
OrderController
OrderService
OrderRepository
OrderEventProducer
OrderCreatedConsumer
NotificationRepository
NotificationController
```

Step 1: Create The Spring Boot Project

Use Spring Initializr or your IDE.

Add dependencies:

- Spring Web
- Spring Data JPA
- Spring for Apache Kafka
- Validation
- H2 Database
- Lombok, optional
- Spring Boot Test

Step 2: Create The Order Entity

```
@Entity
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long customerId;
    private String productName;
    private Integer quantity;
    private String status;
    private LocalDateTime createdAt;

    // getters and setters
}
```

Step 3: Create The Repository

```
public interface OrderRepository extends JpaRepository<Order, Long> {  
}
```

Step 4: Create Request And Response DTOs

```
public record CreateOrderRequest(  
    Long customerId,  
    String productName,  
    Integer quantity  
) {  
}
```

```
public record OrderResponse(  
    Long id,  
    Long customerId,  
    String productName,  
    Integer quantity,  
    String status,  
    LocalDateTime createdAt  
) {  
}
```

Step 5: Create The Event

```
public record OrderCreatedEvent(  
    Long orderId,  
    Long customerId,  
    String productName,  
    Integer quantity,  
    String status,  
    LocalDateTime createdAt  
) {  
}
```

Step 6: Create The Kafka Producer

```
@Service  
public class OrderEventProducer {  
  
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;  
  
    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {  
        this.kafkaTemplate = kafkaTemplate;  
    }  
}
```

```

public void publishOrderCreated(Order order) {
    OrderCreatedEvent event = new OrderCreatedEvent(
        order.getId(),
        order.getCustomerId(),
        order.getProductName(),
        order.getQuantity(),
        order.getStatus(),
        order.getCreatedAt()
    );

    kafkaTemplate.send("order-created", String.valueOf(order.getId()), event);
}
}

```

Step 7: Create The Service

```

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final OrderEventProducer eventProducer;

    public OrderService(OrderRepository orderRepository,
        OrderEventProducer eventProducer) {
        this.orderRepository = orderRepository;
        this.eventProducer = eventProducer;
    }

    public OrderResponse createOrder(CreateOrderRequest request) {
        if (request.customerId() == null) {
            throw new IllegalArgumentException("Customer ID is required");
        }

        if (request.quantity() == null || request.quantity() <= 0) {
            throw new IllegalArgumentException("Quantity must be greater than zero");
        }

        Order order = new Order();
        order.setCustomerId(request.customerId());
        order.setProductName(request.productName());
        order.setQuantity(request.quantity());
        order.setStatus("CREATED");
        order.setCreatedAt(LocalDateTime.now());

        Order saved = orderRepository.save(order);
        eventProducer.publishOrderCreated(saved);

        return toResponse(saved);
    }
}

```

```

public List<OrderResponse> getOrders() {
    return orderRepository.findAll()
        .stream()
        .map(this::toResponse)
        .toList();
}

public OrderResponse getOrder(Long id) {
    Order order = orderRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Order not found"));

    return toResponse(order);
}

private OrderResponse toResponse(Order order) {
    return new OrderResponse(
        order.getId(),
        order.getCustomerId(),
        order.getProductName(),
        order.getQuantity(),
        order.getStatus(),
        order.getCreatedAt()
    );
}
}

```

Step 8: Create The Controller

```

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public OrderResponse createOrder(@RequestBody CreateOrderRequest request) {
        return orderService.createOrder(request);
    }

    @GetMapping
    public List<OrderResponse> getOrders() {
        return orderService.getOrders();
    }

    @GetMapping("/{id}")
    public OrderResponse getOrder(@PathVariable Long id) {
        return orderService.getOrder(id);
    }
}

```

```
}  
}
```

Step 9: Create Notification Persistence

```
@Entity  
public class Notification {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private Long orderId;  
    private String message;  
    private LocalDateTime createdAt;  
  
    // getters and setters  
}
```

```
public interface NotificationRepository extends JpaRepository<Notification, Long> {  
}
```

Step 10: Create Kafka Consumer

```
@Service  
public class OrderCreatedConsumer {  
  
    private final NotificationRepository notificationRepository;  
  
    public OrderCreatedConsumer(NotificationRepository notificationRepository) {  
        this.notificationRepository = notificationRepository;  
    }  
  
    @KafkaListener(topics = "order-created", groupId = "notification-service")  
    public void handleOrderCreated(OrderCreatedEvent event) {  
        Notification notification = new Notification();  
        notification.setOrderId(event.orderId());  
        notification.setMessage("Order created for product: " + event.productName());  
        notification.setCreatedAt(LocalDateTime.now());  
  
        notificationRepository.save(notification);  
    }  
}
```

Step 11: Add Notification Controller

```

@RestController
@RequestMapping("/notifications")
public class NotificationController {

    private final NotificationRepository notificationRepository;

    public NotificationController(NotificationRepository notificationRepository) {
        this.notificationRepository = notificationRepository;
    }

    @GetMapping
    public List<Notification> getNotifications() {
        return notificationRepository.findAll();
    }
}

```

Step 12: Configure Application Properties

```

spring.application.name=module49-lab

spring.datasource.url=jdbc:h2:mem:module49db
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.kafka.bootstrap-servers=localhost:9092

spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer

spring.kafka.consumer.group-id=notification-service
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages=*

```

Step 13: Test With Curl Or Postman

Create an order:

```

curl -X POST http://localhost:8080/orders \
-H "Content-Type: application/json" \
-d '{"customerId":101,"productName":"Laptop","quantity":1}'

```


Get orders:

```
curl http://localhost:8080/orders
```

Get notifications:

```
curl http://localhost:8080/notifications
```

Expected result:

After creating an order, a notification should eventually appear.

Lab Challenge

Add an endpoint:

```
PUT /orders/{id}/status
```

Allowed status flow:

```
CREATED -> APPROVED
CREATED -> CANCELLED
APPROVED -> SHIPPED
```

Then publish another Kafka event:

```
order-status-changed
```

Submission Checklist

- Spring Boot app runs successfully.
- `POST /orders` saves an order.
- `OrderCreatedEvent` is published.
- Kafka consumer receives the event.
- Notification record is created.
- `GET /notifications` returns notification data.
- At least one unit test exists for `OrderService`.
- At least one integration test exists for order creation.

Key Takeaway

Module 49 connects normal backend development with event-driven communication.

```
Spring Boot handles requests and business logic.
Repositories handle persistence.
Kafka lets the system announce important events.
```

Consumers react to those events.
Tests prove the pieces work correctly.